Roscil implement – ai event

Monday, June 29, 2026 11:00 PM

Supervisor Agent

A. Supervisor Agent	
Role	Acts as the brain of the squad. Reads the incoming DisruptionState, classifies severity (LOW/MEDIUM/HIGH/CRITICAL), extracts the affected geography, route, and primary supplier ID from the raw alert text. Routes the classified state to worker agents via LangGraph edges. After all workers complete, compiles the final mitigation brief and decides whether to auto-approve or escalate to HITL.
LLM model	Claude 3 Sonnet (claude-3-sonnet-20240229) — complex multi-step reasoning
Input from DisruptionState	alert_text (String), raw disruption description entered by operator
Output to DisruptionState	severity (Enum), affected_region (String), affected_supplier_id (String), affected_route (Object), routing_decision (String), final_mitigation_brief (Object)
System prompt (condensed)	*You are the Supervisor Agent for ResilioChain, an autonomous supply chain exception management system. Your job: 1. Parse the disruption alert and extract: severity (LOW|MEDIUM|HIGH|CRITICAL), affected_region, affected_supplier_id, affected_route. 2. Update the DisruptionState with your classifications. 3. After worker agents complete, compile a ranked mitigation brief from their outputs. 4. Return structured JSON only. No prose explanations.*
MCP tools called	**read_checkpoint, write_checkpoint, write_plan**
Guardrail / boundary	Cannot proceed to mitigation brief compilation until impacted_orders[], candidate_suppliers[], and compliance_results[] are all populated in DisruptionState. LangGraph conditional edge enforces this — Supervisor node is blocked until all worker nodes return.

Impact Assessor Agent

B. Impact Assessor Agent	
Role	Maps the full blast radius of the disruption. Uses the affected_supplier_id from DisruptionState to run $graphLookup across the Orders → Products → Components → Suppliers dependency graph. Retrieves all active orders that depend on the disrupted supplier or route. Aggregates quantity_remaining across all impacted orders, grouped by SKU category, to compute required_capacity — the minimum supplier capacity needed to fulfil the affected demand.
LLM model	Claude 3 Haiku (claude-3-haiku-20240307) — fast structured data extraction
Input from DisruptionState	affected_supplier_id (String), affected_route (Object) — from Supervisor Agent output in DisruptionState
Output to DisruptionState	affected_skus[] (Array<String>), impacted_orders[] (Array<Object> — each with order_id, sku_id, quantity_remaining, route, status), required_capacity (Number — SUM of quantity_remaining across all impacted_orders[], grouped by sku_category)
System prompt (condensed)	*You are the Impact Assessor Agent for ResilioChain. Given a disrupted supplier ID and route, call the blast_radius_query MCP tool to run $graphLookup on the orders graph. From the returned impacted_orders[], compute: required_capacity = SUM(order.quantity_remaining) for all orders Extract affected_skus[] as the unique list of sku_ids across all impacted orders. Return structured JSON: { affected_skus, impacted_orders, required_capacity } Do not invent order data. Use only what the MCP tool returns.*
MCP tools called	**blast_radius_query**
Guardrail / boundary	May only return supplier data retrieved from the blast_radius_query MCP tool. Cannot fabricate order IDs, SKU names, or quantities. If blast_radius_query returns empty results, returns { affected_skus:[], impacted_orders:[], required_capacity:0 } and halts — does not proceed to Sourcing Agent.

Sourcing Agent

C. Sourcing Agent	
Role	Finds semantically matching backup suppliers. Uses required_capacity (aggregated from impacted_orders[] in DisruptionState by the Impact Assessor) as a hard filter on supplier capacity. Runs $vectorSearch against the suppliers collection (Voyage AI embeddings) to find capability-matched alternatives. Returns ranked candidate suppliers sorted by vector similarity score.
LLM model	Claude 3 Haiku — fast structured data extraction
Input from DisruptionState	affected_skus[] (Array<String>), required_capacity (Number) — both from Impact Assessor output in DisruptionState. Note: required_capacity is NOT fetched externally — it is derived from SUM(impacted_orders[].quantity_remaining)
Output to DisruptionState	candidate_suppliers[] (Array<Object> — each with supplier_id, name, similarity_score, capacity_units_month, geo_region, lead_time_days, esg_score, certifications[])
System prompt (condensed)	*You are the Sourcing Agent for ResilioChain. Given affected_skus[] and required_capacity from DisruptionState, call the supplier_vector_search MCP tool. Pass required_capacity as the minimum capacity filter. The tool returns ranked backup suppliers from MongoDB Atlas Vector Search. Return the top 3 candidates as structured JSON: { candidate_suppliers: [...] } Do not invent suppliers. Return only MCP tool results.*
MCP tools called	**supplier_vector_search**
Guardrail / boundary	Hard-coded to call only supplier_vector_search MCP tool. Cannot return a supplier not present in the Atlas suppliers collection. Capacity filter (capacity_units_month >= required_capacity) is enforced at the $vectorSearch query level — not by the LLM.

Compliance Agent

D. Compliance Agent	
Role	Validates each candidate supplier against enterprise ESG policies, trade sanction lists, labour compliance rules, and carbon-offset requirements. Runs a Vector RAG query against the compliance_policies collection (Voyage AI embeddings of pre-chunked PDF documents) for each candidate. If a supplier fails, LangGraph conditional edge routes back to the Sourcing Agent to try the next candidate. Maximum 3 retries before human escalation.
LLM model	Claude 3 Haiku — fast structured data extraction
Input from DisruptionState	candidate_suppliers[] (one supplier at a time, passed iteratively) — from Sourcing Agent output in DisruptionState
Output to DisruptionState	compliance_result (Object — { supplier_id, passed (Boolean), esg_status, sanctions_status, labour_status, carbon_status, fail_reasons[] }). If passed=true: supplier added to vetted_suppliers[]. If passed=false: fail_reasons[] written to DisruptionState, LangGraph edge triggers retry.
System prompt (condensed)	*You are the Compliance Agent for ResilioChain. For the given supplier, call the compliance_rag_query MCP tool to retrieve relevant ESG, sanctions, and labour policy chunks from the compliance_policies collection. Analyse the retrieved chunks against the supplier's geo_region and certifications[]. Return structured JSON: { supplier_id, passed, esg_status, sanctions_status, labour_status, carbon_status, fail_reasons[] } Be strict. If any policy chunk indicates a risk, set passed=false and list the specific fail_reason. Do not invent policy content. Use only what the MCP tool returns.*
MCP tools called	**compliance_rag_query**
Guardrail / boundary	Compliance guardrail node. If passed=false: LangGraph conditional edge routes back to Sourcing Agent (max 3 retries). The mitigation brief cannot be compiled until at least one supplier has passed=true. If all 3 retries fail, escalates to HITL with fail_reasons[] displayed in Streamlit UI.

Mitigation Planner

E. Mitigation Planner (Supervisor — second pass)	
Role	Final node in the LangGraph graph. Reads the complete DisruptionState (impacted_orders[], vetted_suppliers[], compliance_results[], cost data) and compiles a ranked mitigation brief. Calculates cost variance between original and backup supplier. Assigns a confidence score. Writes the plan to the mitigation_plans collection via MCP. Determines whether auto-approve (confidence >= threshold) or HITL review is required.
LLM model	Claude 3 Sonnet — complex multi-step reasoning, plan compilation
Input from DisruptionState	All DisruptionState fields populated by previous agents: affected_skus[], impacted_orders[], required_capacity, vetted_suppliers[], compliance_results[]
Output to DisruptionState	mitigation_brief (Object — { plan_id, affected_orders_count, backup_supplier, reroute_via, lead_time_delta_days, cost_variance_usd, cost_variance_pct, esg_status, confidence_score, recommended_action, options[] })
System prompt (condensed)	*You are the Mitigation Planner for ResilioChain. You have the complete DisruptionState with all agent outputs. Compile a mitigation brief: 1. Select the top vetted supplier from vetted_suppliers[]. 2. Calculate cost_variance = (backup_supplier.unit_price - original.unit_price) * required_capacity. 3. Assign confidence_score (0-100) based on: supplier similarity score, ESG rating, capacity margin. 4. Set recommended_action: AUTO_EXECUTE if confidence >= 85, else HUMAN_REVIEW. Return structured JSON only. Write to mitigation_plans collection via write_plan MCP tool.*
MCP tools called	**write_plan, update_plan_status, read_checkpoint**
Guardrail / boundary	Cannot compile plan until vetted_suppliers[] contains at least one entry with passed=true. Confidence score calculation is deterministic (not LLM-generated) — formula enforced in tool schema. HITL is mandatory if confidence < 85 or if any vetted supplier has esg_score < 60.